

Feature Specification: Documentation Agent Pipeline

Generated from specs/001-documentation-agent/spec.md

Artifact type: spec

Feature Branch: `001-documentation-agent`

Table of contents

Feature Specification: Documentation Agent Pipeline

User Scenarios & Testing **(mandatory)**

User Story 1 - Generate documentation from repository artifacts (Priority: P1)

User Story 2 - Avoid duplicate work and preserve version history (Priority: P2)

User Story 3 - Maintain readable, structured documentation output (Priority: P2)

Edge Cases

Requirements **(mandatory)**

Functional Requirements

Key Entities **(include if feature involves data)**

Success Criteria **(mandatory)**

Measurable Outcomes

Assumptions

Body

Feature Specification: Documentation Agent Pipeline

Feature Branch: `001-documentation-agent` **Created**: 2026-07-16 **Status**: Draft

This feature is to be delivered as a Spec Kit extension package. Its project directory should be maintained in parallel to the Spec Kit source tree, rather than being embedded inside the core Spec Kit implementation. **Input**: User description: "Read docs-agent-feature-spec.md in this repo. Use §1 (Purpose) and §6 (Pipeline Detail) as the specification for what to build — describe the discovery, classification, dedupe, validation, and render behavior in your own spec.md, in your own words, based on what's there. Do not include any tech stack decisions here (that belongs in /speckit.plan) and do not reference §9 or the Kiro-related parts of §0 — the frontend is being built separately by a different tool in a later phase and is explicitly out of scope for this spec."

User Scenarios & Testing *(mandatory)*

> Constitutional constraints for this feature: the implementation **MUST** remain within the Spec Kit-side extension/backend/pipeline scope, preserve the zero-config agent-native path, and avoid the listed non-goals such as frontend work and unrelated product features.

User Story 1 - Generate documentation from repository artifacts (Priority: P1)

A developer wants the system to turn the repository's Markdown spec artifacts into organized, browsable documentation without manually copying or reformatting them. ****Why this priority****: This is the core value of the feature and the minimum useful outcome.

****Independent Test****: A developer can trigger documentation generation for an existing set of markdown artifacts and receive a document version that reflects those source files.

****Acceptance Scenarios****: 1. ****Given**** a repository containing markdown artifacts in the supported spec tree, ****When**** the documentation workflow is triggered, ****Then**** the system discovers those files and prepares them for processing. 2. ****Given**** a markdown artifact that is recognized by the classification logic, ****When**** it is ingested, ****Then**** it is assigned the appropriate document role and included in the generated documentation. ---

User Story 2 - Avoid duplicate work and preserve version history (Priority: P2)

A developer wants the system to avoid reprocessing the same content repeatedly and to keep a clear record of each successful rendering. ****Why this priority****: Reprocessing and duplicate records would create noise and reduce trust in the documentation. ****Independent Test****: When the same content is submitted again without changes, the system returns the existing result and does not create a duplicate document version. ****Acceptance Scenarios****: 1.

****Given**** an artifact that has already been processed, ****When**** the same content is submitted again, ****Then**** the system detects that the content has not changed and does not create a new version. 2. ****Given**** an artifact that has changed since the last successful render, ****When**** the workflow runs again, ****Then**** the system treats it as new work and creates a fresh version. ---

User Story 3 - Maintain readable, structured documentation output (Priority: P2)

A reviewer wants the generated documentation to be easy to scan and understand, with distinct sections for tasks, stories, decisions, and other content types. The implementation now includes an explicit agent-style transformation step that inspects markdown content, derives titles and abstracts, groups content into section types, and prepares a user-friendly structure before PDF rendering. ****Why this priority****: The value of the system depends not just on conversion, but on the quality and structure of the resulting documentation.

****Independent Test****: A generated document can be inspected and clearly shows the intended sections and content types rather than a flat dump of markdown. ****Acceptance Scenarios****: 1. ****Given**** a source artifact with task-oriented content, ****When**** it is rendered, ****Then**** it appears as a distinct section type in the output rather than as ordinary prose. 2. ****Given**** an artifact with content that does not match a known category, ****When**** it is processed, ****Then**** it is still ingested and rendered using a generic fallback category. ---

Edge Cases

- What happens when a markdown artifact is empty or missing required content? - How does the system handle files that do not match a recognized role? - What happens when a change arrives through a fallback path that cannot produce structured content immediately? - What happens when validation rejects generated output from the agent-native path?

Requirements **(mandatory)**

Functional Requirements

- ****FR-001****: The system MUST discover markdown artifacts from the supported repository spec locations and make them available for processing. - ****FR-002****: The system MUST classify each discovered markdown artifact into a recognized document role when possible and use a generic fallback role for anything unrecognized. - ****FR-003****: The system MUST compute a content fingerprint for each artifact and use it to detect whether the content has changed since the last successful processing. - ****FR-004****: The system MUST prevent duplicate processing when the current content matches the most recent known version for the same artifact. - ****FR-005****: The system MUST create a new documented version when an artifact's content has changed and the content can be validated for rendering. - ****FR-006****: The system MUST validate incoming content before producing a new render and must not silently accept malformed or incomplete data. - ****FR-006A****: When validation rejects generated output, the system MUST stop immediately, return the specific missing headings or misclassified sections that caused the failure, and leave retry decisions to the calling agent rather than retrying automatically. - ****FR-007****: The system MUST preserve a versioned record of rendered documentation so that each successful generation is traceable. - ****FR-008****: The system MUST render the documentation in an organized structure that distinguishes common artifact types such as tasks, user stories, design decisions, and other narrative content. - ****FR-008A****: The pipeline MUST include an explicit agent-style transformation stage that interprets markdown into a structured document plan with a title, abstract, and grouped sections before PDF generation. - ****FR-008B****: The generated document MUST include a polished cover page, a table of contents, grouped section blocks, and footer metadata so the output is user-friendly rather than a flat markdown dump. - ****FR-009****: The system MUST retain an artifact record even when its content does not fit a known category, rather than dropping it from the workflow. - ****FR-010****: When content changes through a fallback pathway that cannot immediately produce fully structured renderable content, the system MUST mark the artifact as needing regeneration rather than pretending it is fully rendered.

Key Entities **(include if feature involves data)**

- ****Artifact****: A markdown file discovered from the supported spec locations, with content, classification, and processing state. - ****Document Version****: A rendered output record tied to a specific artifact and representing one successful generation of documentation. - ****Project****: The repository or work product that owns the artifacts and their documentation history.

Success Criteria **(mandatory)**

Measurable Outcomes

- **SC-001**: 100% of supported markdown artifacts discovered in the target repository locations are either processed successfully or explicitly marked for follow-up when they cannot yet be rendered. - **SC-002**: Repeated submissions of unchanged content do not create duplicate document versions. - **SC-003**: A changed artifact produces a new documented version the next time the workflow runs with valid content. - **SC-004**: Generated documentation clearly separates core section types such as tasks and user stories from ordinary narrative content. - **SC-005**: Users can identify the latest renderable version for each artifact and understand whether an artifact is pending, rendered, or needs regeneration. - **SC-006**: The pipeline produces a user-friendly document structure from raw markdown through the agent-style transformation stage rather than emitting a flat, ungrouped conversion. - **SC-007**: The generated PDF presents a polished layout with cover-page metadata, a table of contents, grouped sections, and footer details that make the artifact easier to review.

Assumptions

- The feature operates on markdown artifacts already stored in the repository's supported spec locations. - The deliverable is a Spec Kit extension, and its project directory should sit alongside the Spec Kit source code rather than inside it. - The scope covers discovery, classification, deduplication, validation, and rendering behavior; it does not include building the later frontend experience. - The system is expected to handle both recognized and unrecognized artifact categories by assigning a fallback role rather than ignoring them. - The workflow may receive content through different entry points, but all of them must preserve the same core validation and versioning rules.

source: specs/001-documentation-agent/spec.md | commit: n/a